Reviewer Pack — Algebraic Hodge Theory and Resolution Proof Complexity

Entry 7f29c18cff7c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 13:44:58 UTC

Algebraic Hodge Theory and Resolution Proof Complexity

Entry ID: `7f29c18cff7c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 13:44:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

> For any CNF formula φ, the Hodge polynomial of its associated projective variety V(φ) has a degree that is linearly correlated with the resolution proof width of φ, such that deg(H(V(φ))) = Θ(w(φ)) for all CNFs φ.

Rationale (proposer's reasoning):

> Algebraic Hodge theory provides a rich set of invariants for studying algebraic varieties. By exploring the relationship between these invariants and resolution proof complexity, we may uncover new structural insights into the hardness of solving SAT instances. The Hodge polynomial has not been widely used in this context, making this conjecture novel.

Taxonomy category: `HODGE_THEORY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `58ff6ac70a831bed`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of the 30 generated CNFs with n ≤ 40 variables, the correlation coefficient between deg(H(V(φ))) and w(φ) is ≥ 0.7. The conjecture is falsified if this condition is not met.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge polynomial" AND "resolution proof complexity"` - `"Algebraic geometry" AND "CNF formula resolution width"` - `"degree of Hodge polynomial" related TO "proof complexity"`

Top relevant hits considered: - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/1607.00443v1] Algebraic Proof Complexity: Progress, Frontiers and Challenges - [http://arxiv.org/abs/1711.07320v2] Proof Complexity Meets Algebra

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.7s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        if A[i][i] == 0:
            # Find a non-zero pivot below the current row
            for k in range(i + 1, n):
                if A[k][i] != 0:
                    # Swap rows i and k
                    A[i], A[k] = A[k], A[i]
                    break
            else:
                raise ValueError("Matrix is singular")
        for j in range(n):
            if j == i:
                continue
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def projective_variety(phi):
    n = len(phi)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(n):
            if phi[i][j]:
                A[i][j] = 1
    return gaussian_elimination(A)
```

```
def resolution_width(phi):
    n = len(phi)
    clauses = [set(range(n)) for _ in range(n)]
    assignment = {}

    def dpll():
        if not any(clause for clause in clauses):
            return True
        unit_clause = next((clause for clause in clauses if len(clause) == 1), None)
        if unit_clause:
            literal = list(unit_clause)[0]
            assignment[literal] = True
            new_clauses = []
            for clause in clauses:
                if literal in clause:
                    continue
                if -literal in clause:
                    return False
                new_clauses.append(clause - {literal})
            clauses[:] = new_clauses
        else:
            literal = next(lit for lit in range(n) if lit not in assignment)
            assignment[literal] = True
            if dpll():
                return True
            del assignment[literal]
            assignment[-literal] = True
            if dpll():
                return True
            del assignment[-literal]
        return False

    return len(assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    phi = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    try:
        variety = projective_variety(phi)
        width = resolution_width(phi)
        deg = sum(len(row) - row.count(0) for row in variety if any(x != 0 for x in row))

        return {
            "metric_name": "Hodge degree",
            "metric_value": deg,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": deg == width,
            "counterexample": ""
        }
    except Exception as e:
        return {
            "metric_name": "Hodge degree",
```

```
            "metric_value": None,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": str(e)
        }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results if res["metric_value"] is not None) /
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results if res["metric_val
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={f
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds':
TRIAL: {"seed": 593, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 631, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 677, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 727, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 773, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 821, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 877, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,
TRIAL: {"seed": 929, ...{'metric_name': 'Hodge degree', 'metric_value': None, 'instances_tested': 1,

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d4e9c3a.py", line 128, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={first_f
                                                ^^^
NameError: name 'res' is not defined. Did you mean: 're'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test failed to produce a valid result due to a crash, indicating that the conjecture does not hold for at least one instance tested. | next: Investigate the cause of the crash and attempt to replicate the counterexample. If the crash is resolved, retest with additional instances to confirm or refute the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13527
2	preregistration	ollama_remote	glm4:latest	0	0	12231
3	novelty	ollama_remote	glm4:latest	0	0	12947
4	novelty	ollama_remote	glm4:latest	0	0	11141
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12812
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	68116
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	60258
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17809
9	judge	ollama_remote	glm4:latest	0	0	44915

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 253756 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7f29c18cff7c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7f29c18cff7c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/7f29c18cff7c.tar.gz` (if generated)